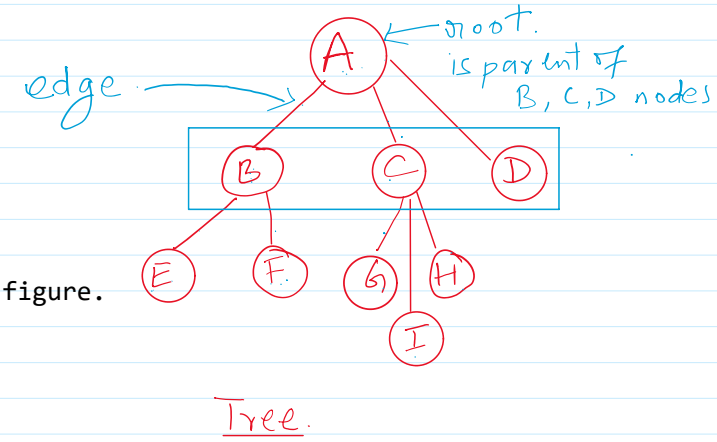
Trees and Binary Tree

11 May 2026 19:58

What is Tree in Computer Science?
A **Tree** is a non-linear data structure used to store hierarchical data.

Unlike arrays or linked list, tree represent data in a parent-child relationship.

Example:
- File systems
- Organizational hierarchy
- XML/HTML structure
- Database indexing



Tree.

Root Node: The topmost node of a tree.
Example: A is the root node in the given figure.

Parent Node: A node having child nodes
Example: A is a parent of B, C, D.

Child Node: Nodes connected below a parent node.
Example: B, C, D are child nodes of A.

Leaf node: Nodes having no children
Example: E, F, G, I, H, D in our diagram.

Siblings: Node having same parent.
Example: B, C, D are siblings having A as parent.

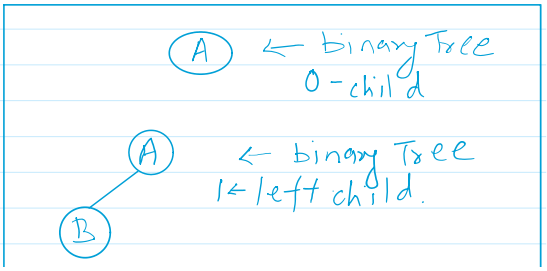
Edge: Connection between two nodes
Example: A-----B

Height of Tree: Maximum number of edges from root to deepest node.
Example: Height = 2 meaning Maximum number of edges from root to deepest node in our diagram.

Types of Trees
- **General Tree**
- **Binary Tree**
- **Binary Search Tree**
- **AVL tree**
- **Heap**
- **B-Tree**
- **Expression Tree**
- **Recursion Tree**
- **Red-Black Tree**
- **And many more.**

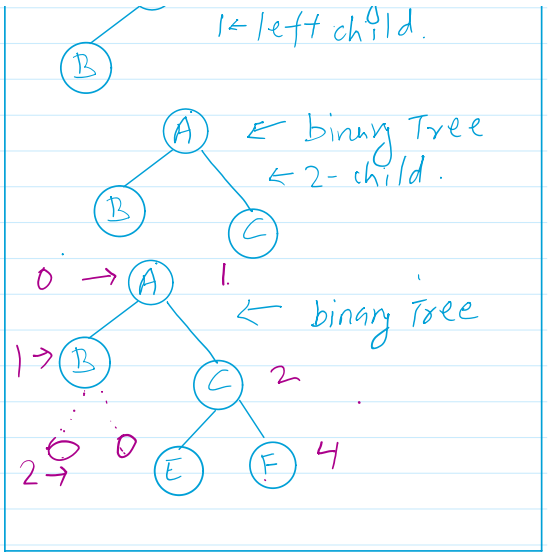
Binary Tree

A **Binary Tree** is a tree in which each node can have at most:
- One left child
- One Right child
- Or No-child



Properties of Binary Tree.

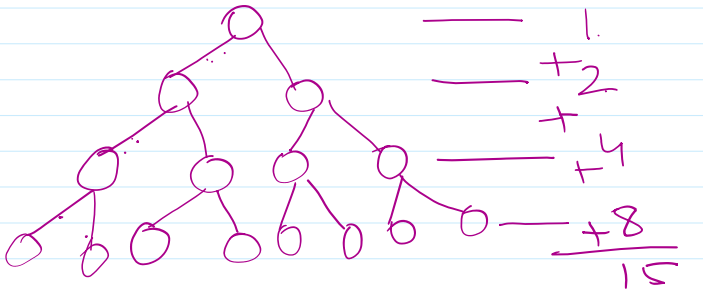
Properties of Binary Tree.



Structures of Binary Tree.

⊛ Maximum nodes at level $L = 2^L$

⊛ Maximum number of nodes in tree of height h.

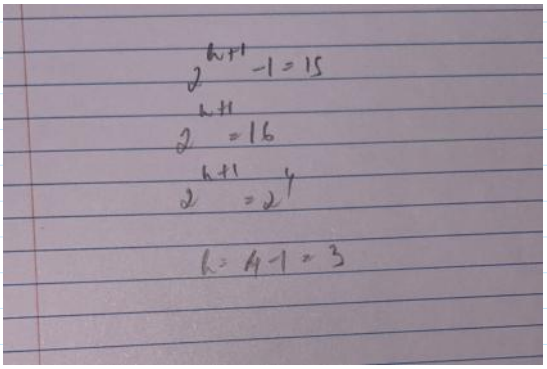


$= 2^{h+1} - 1$ [$\because h$ = height].

$= 2^{3+1} - 1 = 16 - 1 = 15$

⊛ Minimum height of binary tree with $n = 15$ node.

$2^{h+1} - 1 = 15$ find h.



$2^{h+1} - 1 = n$.

or, $2^{h+1} = n+1$

put $\log_2$ both side.

or $\log_2 2^{(h+1)} = \log_2(n+1)$

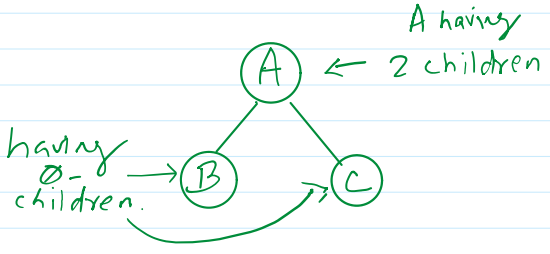
or $(h+1)\log_2 2 = \log_2(n+1)$

or $h+1 = \log_2(n+1)$

or $h = \log_2(n+1) - 1$

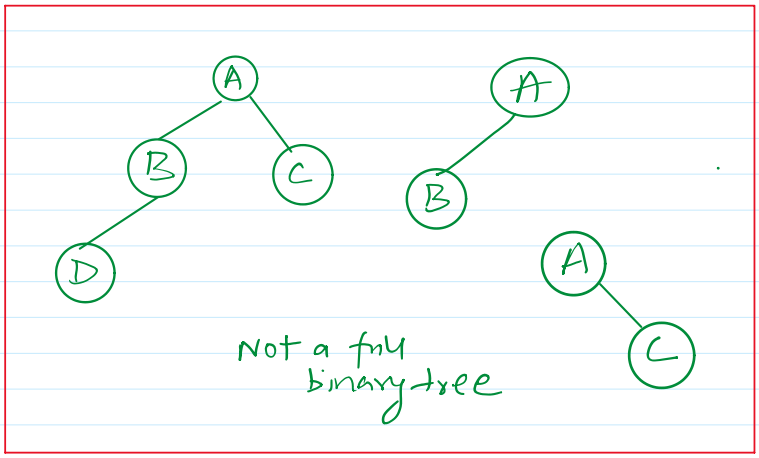
$$h_{min} = \log_2(n+1) - 1$$

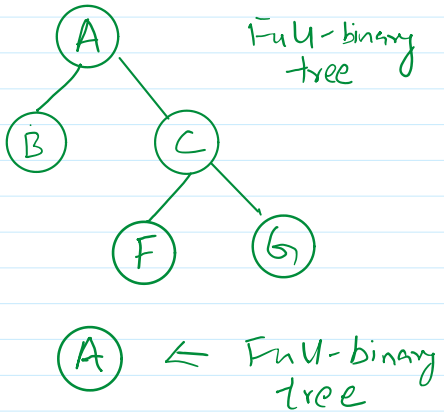
Types of Binary Trees



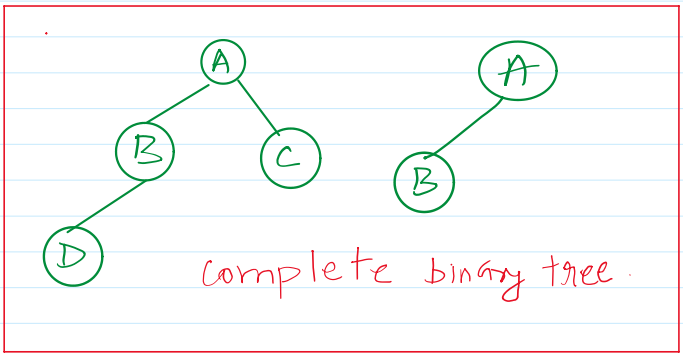
1. Full Binary Tree: Every node has either:

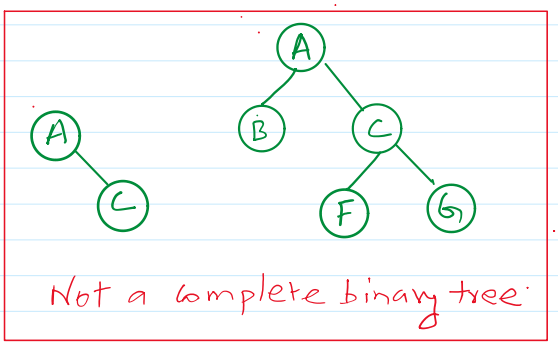
- 0-children
- Or 2-children



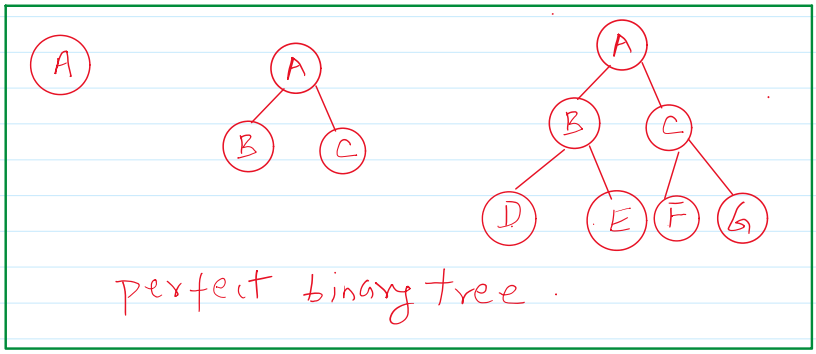


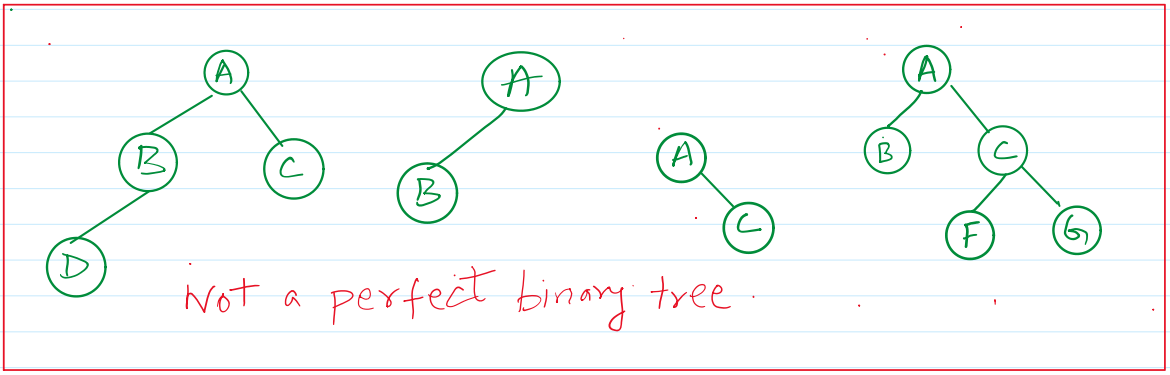
2. Complete Binary Tree: All levels completely filled except possibly last level.
Last level filled from left to right.



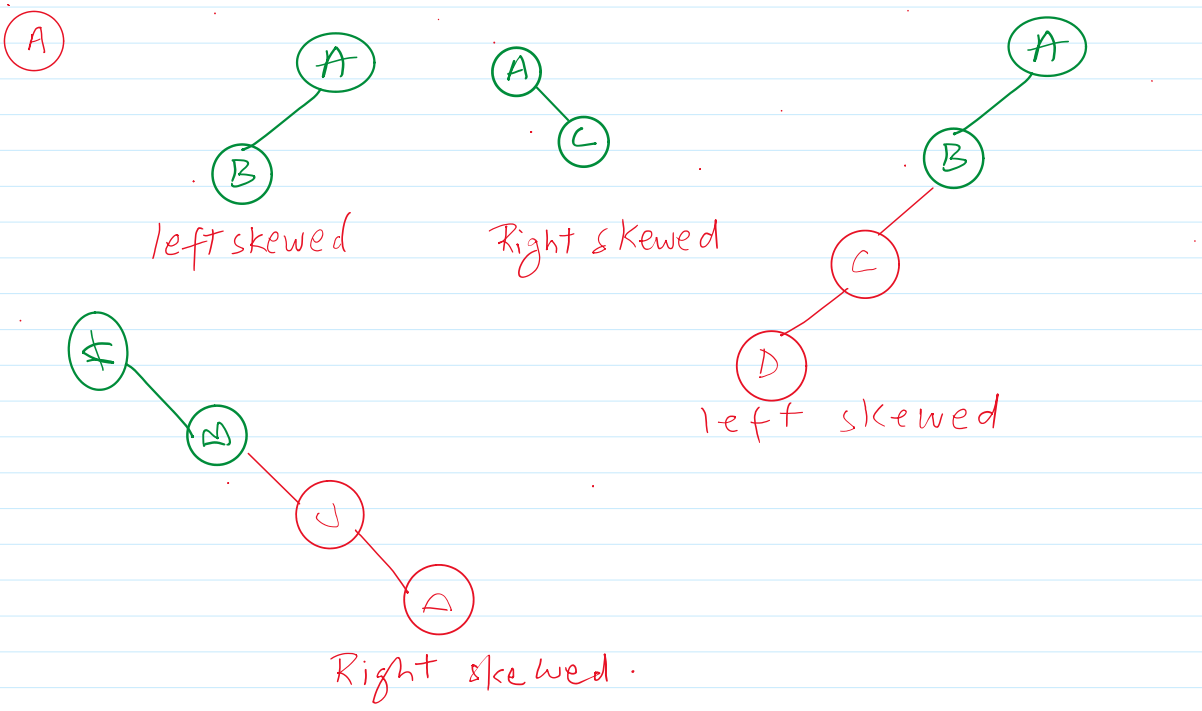


3. Perfect Binary Tree: All internal nodes have 2 children and all leaf nodes are at same level.

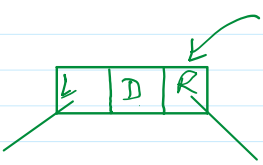


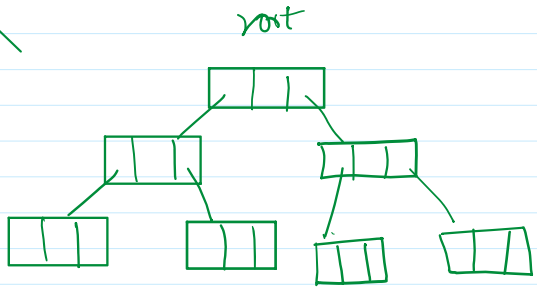


4. Degenerated Binary Tree: Each parent has only one child, it looks like linked list.

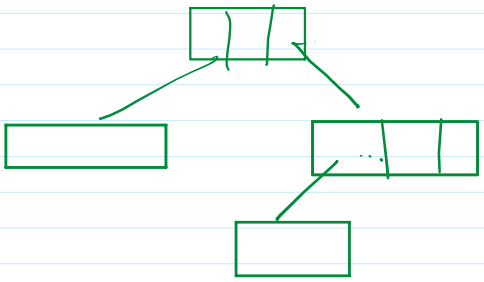



```
class Node {
    int data;
    Node left, right;
    ⋮
    Node(){}.
    Node(data){}.
    set and get methods
}
class BinaryTree {
```





Doubly Linked list logically hierarchical is called binary tree structure.



```
package BinaryTree;
public class Node {
```

```
    private int data;
    private Node left, right;
    public Node(){
        data = 0;
        left = right = null;
    }
    public Node(int data) {
        this.data = data;
    }
    public int getData() {
        return data;
    }
    public void setData(int data) {
        this.data = data;
    }
    public Node getLeft() {
        return left;
    }
    public void setLeft(Node left) {
        this.left = left;
    }
    public Node getRight() {
        return right;
    }
    public void setRight(Node right) {
        this.right = right;
    }
    @Override
    public String toString() {
        return "Node [data=" + data + "]";
    }
}


package BinaryTree;
import java.util.Scanner;
public class BinaryTreeDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Node root = new Node(100);
        Node left = new Node();
        System.out.println("Enter data for node: ");
        left.setData(sc.nextInt());
        root.setLeft(left);
        Node right = new Node();
        System.out.println("Enter data for node: ");
        right.setData(sc.nextInt());
        root.setRight(right);

        // System.out.println("Root node = " + root);
        // System.out.println("Left child = "+left);
        // System.out.println("Right child = " + right);

        System.out.println("Root node = " + root);
        System.out.println("Left child = "+root.getLeft());
        System.out.println("Right child = " + root.getRight());
        Node rightLeft = new Node(150);
        right.setLeft(rightLeft);
        System.out.println("Adding a node on the left side of right child:");
        System.out.println("Root node = " + root);
```

```
        System.out.println("Left child = "+root.getLeft());
        System.out.println("Right child = " + root.getRight());
        System.out.println("Left of Right Child = " + root.getRight().getLeft());
    }
}
```

```
Output:
   Enter data for node:
   50
   Enter data for node:
   200
   Root node = Node [data=100]
   Left child = Node [data=50]
   Right child = Node [data=200]
   Adding a node on the left side of right child:
   Root node = Node [data=100]
   Left child = Node [data=50]
   Right child = Node [data=200]
   Left of Right Child = Node [data=150]
```

Tree Traversal

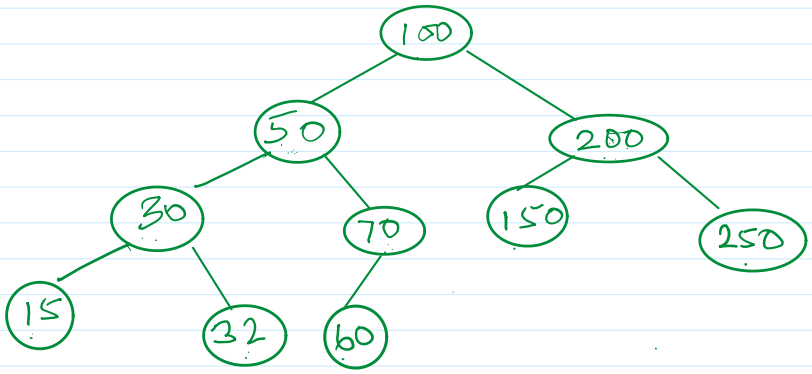
Traversal means visiting all nodes

There are four major traversals:

1. Preorder
2. Inorder
3. Postorder
4. Level Order

1. **Preorder traversal**

   Root->Left->Right



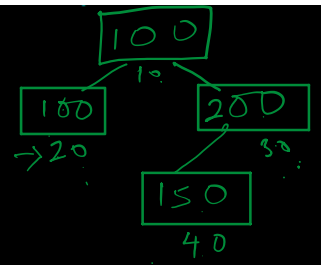
100, 50, 30, 15, 32, 70, 60, 200, 150, 250

```
package BinaryTree;
import java.util.Scanner;
public class BinaryTreeDemo {
    public static void preorder(Node root){
        if(root == null){
            return;
        }
        System.out.print("==>"+root.getData());
        preorder(root.getLeft());
        preorder(root.getRight());
    }
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Node root = new Node(100);
        Node left = new Node();
        System.out.println("Enter data for node: ");
        left.setData(sc.nextInt());
        root.setLeft(left);
        Node right = new Node();
        System.out.println("Enter data for node: ");
        right.setData(sc.nextInt());
        root.setRight(right);

        // System.out.println("Root node = " + root);
        // System.out.println("Left child = "+left);
        // System.out.println("Right child = " + right);

        System.out.println("Root node = " + root);
        System.out.println("Left child = "+root.getLeft());
        System.out.println("Right child = " + root.getRight());
        Node rightLeft = new Node(150);
        right.setLeft(rightLeft);
        System.out.println("Adding a node on the left side of right child:");
        System.out.println("Root node = " + root);
        System.out.println("Left child = "+root.getLeft());
        System.out.println("Right child = " + root.getRight());
        System.out.println("Left of Right Child = " + root.getRight().getLeft());
        System.out.println("\nPreorder traversal: ");
        preorder(root);
    }
}
```

Output:

```
Enter data for node:
100
Enter data for node:
200
Root node = Node [data=100]
Left child = Node [data=100]
Right child = Node [data=200]
Adding a node on the left side of right child:
Root node = Node [data=100]
Left child = Node [data=100]
Right child = Node [data=200]
Left of Right Child = Node [data=150]

Preorder traversal:
==>100==>100==>200==>150
```




```
public static void preorder(Node root){
        if(root == null){
            return;
        }
        System.out.print("==>"+root.getData());
        preorder(root.getLeft());
        preorder(root.getRight());
    }
```

due

preorder(10.getRight())

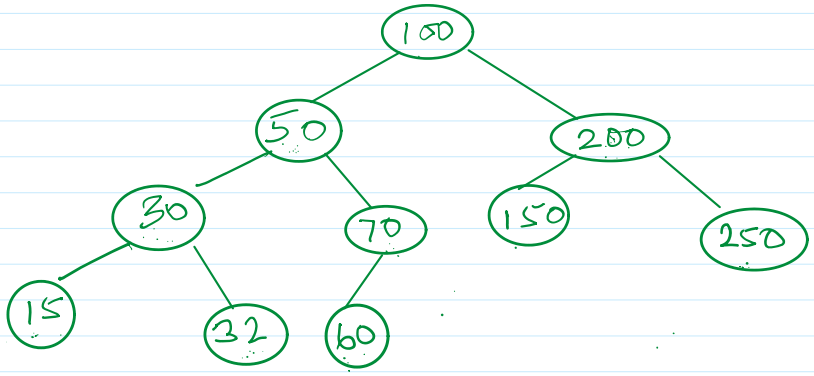
preorder(20.getRight())

preorder(30.getRight())

preorder(40.getRight())

2. Inorder traversal

Left->Root->Right



15, 30, 32, 50, 60, 70, 100, 150, 200, 250

```
public static void inorder(Node root){
        if(root == null){
            return;
        }
      ->inorder(root.getLeft());
        System.out.print("==>"+root.getData());
        inorder(root.getRight());
    }
```

due:

print(10.getData())
inorder(10.getRight())

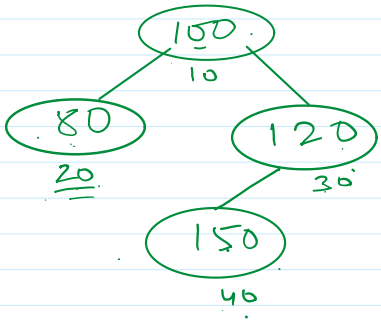
print(20.getData())
inorder(20.getRight())

print(30.getData())
inorder(30.getRight())

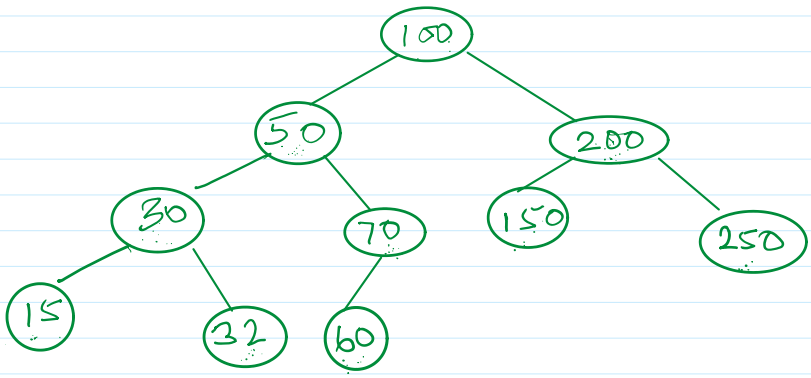
print(40.getData())
inorder(40.getRight())

Output:

```
Enter data for node:
80
Enter data for node:
120
Root node = Node [data=100]
Left child = Node [data=80]
Right child = Node [data=120]
Adding a node on the left side of right child:
Root node = Node [data=100]
Left child = Node [data=80]
Right child = Node [data=120]
Left of Right Child = Node [data=150]

Preorder traversal:
==>100==>80==>120==>150
Inorder traversal:
==>80==>100==>150==>120
```



3. Postorder traversal
Left->Right->Root



15, 32, 30, 60, 70, 50, 150, 250, 200, 100.

```
public static void postorder(Node root){
        if(root == null){
            return;
        }
        postorder(root.getLeft());
        postorder(root.getRight());
        System.out.print("==>"+root.getData());
    }
```

due:

postorder(10.getRight())
print(10.getData())

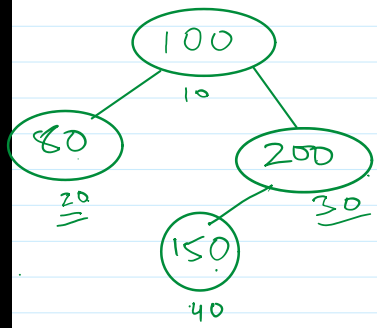
postorder(20.getRight())
print(20.getData())

postorder(30.getRight())
print(30.getData())

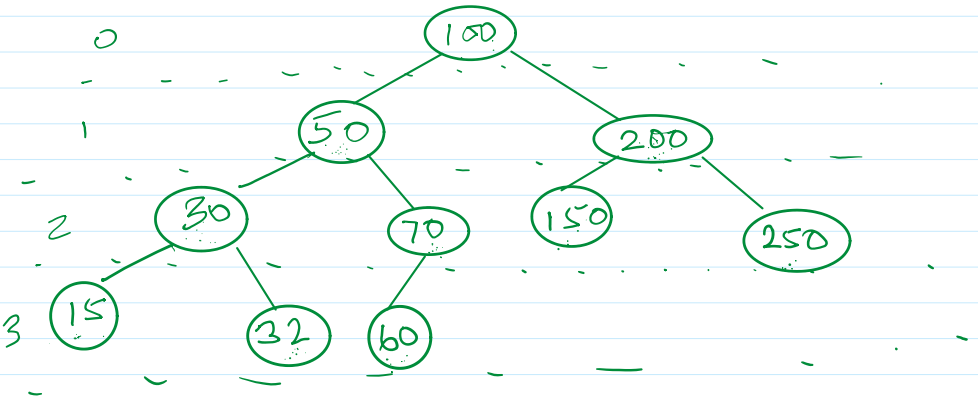
postorder(40.getRight())
print(40.getData())

Output:

```
Enter data for node:
80
Enter data for node:
200
Root node = Node [data=100]
Left child = Node [data=80]
Right child = Node [data=200]
Adding a node on the left side of right child:
Root node = Node [data=100]
Left child = Node [data=80]
Right child = Node [data=200]
Left of Right Child = Node [data=150]

Preorder traversal:
==>100==>80==>200==>150
Inorder traversal:
==>80==>100==>150==>200
Postorder traversal:
==>80==>150==>200==>100
```



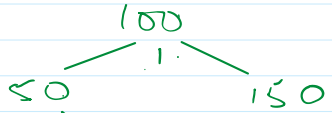
4. Level order traversal:



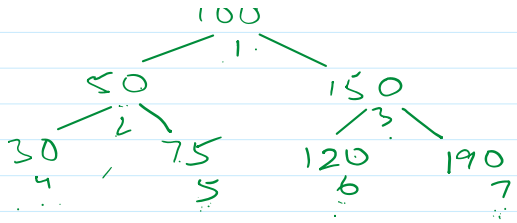
100, 50, 200, 30, 70, 150, 250, 15, 32, 60

Level order traversal visits nodes level by level from top to bottom and left to right. It uses a Queue data structure.

```
public static void levelorder(Node root){
        if (root == null){
```



```java
public static void levelorder(Node root){
        if (root == null){
            return;
        }
        Queue<Node> q = new LinkedList<>();
        System.out.println();
        q.add(root);
        while(!q.isEmpty()){
            Node t = q.poll();
            System.out.print("==>"+t.getData());
            if(t.getLeft()!= null){
                q.add(t.getLeft());
            }
            if(t.getRight()!= null){
                q.add(t.getRight());
            }
        }
    }
```



q = X 2, 3, 4, 5, 6, 7

t = X 2 3 4 5 6 7

output:

==> 100 ==> 50 ==> 150 ==> 30 ==> 75,
==> 120 ==> 190.